

MACHINE LEARNING
ANALISIS & IMPLEMENTASI *DECISION TREE* & *NAIVE BAYES*



Disusun Oleh:

Nurmelizah – G1A024013

Tasya Triani – G1A024019

Lovien Najla Dhafiyah – G1A024055

Dosen Pengampu:

Ir. Arie Vatresia, S.T., M.T.I., Ph.D., IPP.

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU

2026

BAGIAN A – ANALISIS KONSEP

1. Jelaskan kapan *Decision Tree* lebih cocok digunakan dibanding *Naive Bayes* untuk kasus prediksi kelulusan ini. Beri 2 alasan!

Jawab: *Decision Tree* lebih cocok digunakan pada dataset prediksi kelulusan ini karena sifat hubungannya yang kompleks dan kebutuhan interpretasi manajerial:

- a. Mampu Menangkap Interaksi *Non-Linear* Antar-Fitur. Pada dunia akademik nyata, pengaruh fitur sering kali bersifat kondisional, bukan independen. Sebagai contoh, jam belajar yang tinggi mungkin hanya berdampak positif jika mahasiswa memiliki tingkat kehadiran yang memenuhi batas minimal perkuliahan. *Decision Tree* membagi ruang fitur secara hierarkis (bertingkat) sehingga mampu memodelkan aturan kombinasi variabel (*if-else logic*) tanpa memerlukan asumsi distribusi tertentu.
 - b. Transparansi dan Kebutuhan *Explainability* (XAI). Dalam domain pendidikan (*EduTech* dan perguruan tinggi), keputusan institusional (seperti peringatan dini akademik atau konseling) memerlukan alasan logis yang dapat dipertanggungjawabkan kepada mahasiswa dan dosen wali. *Decision Tree* menghasilkan representasi diagram visual pohon keputusan yang langsung memperlihatkan batas ambang batas (*splitting threshold*, misal: Kehadiran 75% dan IPK 2.75), sedangkan *Naive Bayes* bekerja secara probabilistik agregat yang sulit ditelusuri urutan pemicu utamanya.
2. Asumsi “independen” di *Naive Bayes*. Sebutkan 2 pasang fitur di dataset yang kemungkinan melanggar asumsi ini dan jelaskan kenapa!

Jawab: Algoritma *Naive Bayes* mengasumsikan bahwa seluruh prediktor bersifat saling bebas secara bersyarat (*conditional independence*) terhadap variabel target. Pada dataset mahasiswa_lulus.csv, terdapat 2 pasang fitur yang melanggar asumsi ini:

- a. Pasangan IPK dan Status_Beasiswa. Mahasiswa penerima beasiswa (*Status_Beasiswa = Ya*) pada umumnya diwajibkan mempertahankan standar akademik dengan batas nilai IPK minimum tertentu (misalnya IPK 3.00 atau 3.25). Akibatnya, distribusi nilai IPK pada kelompok penerima beasiswa terkonsentrasi di batas atas, membuktikan adanya korelasi dan ketergantungan yang kuat antar-kedua variabel tersebut.
- b. Pasangan Kehadiran dan Jam_Belajar. Mahasiswa yang memiliki kedisiplinan belajar mandiri tinggi (*Jam_Belajar* tinggi) cenderung menunjukkan komitmen akademik yang sejalan dengan tingkat presensi kuliah yang tinggi. Sebaliknya, mahasiswa dengan jam belajar minim kerap berkorelasi dengan angka absensi perkuliahan yang buruk. Adanya multikolinearitas ini menyebabkan pembobotan ganda (*double-counting of evidence*) pada perhitungan probabilitas posterior *Naive Bayes*.

3. Apa risiko overfitting pada *Decision Tree* di dataset ini? Bagaimana cara mengatasinya?

Jawab: Jika *Decision Tree* dibiarkan tumbuh tanpa batas (`max_depth = None`), pohon akan membelah data sampai daun terakhir demi mengejar skor sempurna di data latih. Akibatnya, pada dataset kecil (500 data) yang punya banyak faktor acak, model justru “menghafal” data latih beserta erornya, sehingga performanya langsung turun drastis saat menguji data baru. Cara mengatasinya adalah dengan membatasi pertumbuhan pohon sejak awal (*pre-pruning*), misalnya mengunci kedalaman pohon (`max_depth = 3`) atau menentukan syarat minimal sampel di setiap daun. Selain itu, bisa juga dengan memangkas cabang yang tidak penting setelah pohon jadi (*post-pruning*), atau menggunakan metode gabungan seperti *Random Forest*.

BAGIAN B – IMPLEMENTASI & EVALUASI

Bagian B0 – Persiapan Data & Import Library (Inisialisasi Dataset)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

np.random.seed(42)

jumlah_data = 500

ipk = np.random.normal(3.0, 0.4, jumlah_data)
ipk = np.clip(ipk, 2.0, 4.0)

kehadiran = np.random.normal(80, 12, jumlah_data)
kehadiran = np.clip(kehadiran, 40, 100)

jam_belajar = np.random.normal(15, 7, jumlah_data)
jam_belajar = np.clip(jam_belajar, 0, 40)
```

```
organisasi = np.random.choice(['Aktif', 'Tidak Aktif'], jumlah_data, p=[0.4, 0.6])
penghasilan_ortu = np.random.choice(['Rendah', 'Menengah', 'Tinggi'], jumlah_data, p=[0.3, 0.5, 0.2])
jenis_kelamin = np.random.choice(['L', 'P'], jumlah_data)
beasiswa = np.random.choice(['Ya', 'Tidak'], jumlah_data, p=[0.25, 0.75])

skor = (-3
        + 4 * ((ipk - 2) / 2)
        + 3 * ((kehadiran - 40) / 60)
        + 1.5 * (jam_belajar / 40)
        + np.random.normal(0, 1.0, jumlah_data))

peluang_lulus = 1 / (1 + np.exp(-skor))
lulus_tepat_waktu = np.where(np.random.rand(jumlah_data) < peluang_lulus, 'Ya', 'Tidak')

data = pd.DataFrame({
    'IPK': ipk.round(2),
    'Kehadiran': kehadiran.round(1),
    'Jam_Belajar': jam_belajar.round(1),
    'Organisasi': organisasi,
    'Penghasilan_Ortu': penghasilan_ortu,
    'Jenis_Kelamin': jenis_kelamin,
    'Status_Beasiswa': beasiswa,
    'Lulus_Tepat_Waktu': lulus_tepat_waktu
})
```

```
kolom_numerik = ['IPK', 'Kehadiran', 'Jam_Belajar']
for kolom in kolom_numerik:
    baris_kosong = np.random.choice(data.index, size=int(0.05 * jumlah_data), replace=False)
    data.loc[baris_kosong, kolom] = np.nan

data.to_csv('mahasiswa_lulus.csv', index=False)

data.head()
```

	IPK	Kehadiran	Jam_Belajar	Organisasi	Penghasilan_Ortu	Jenis_Kelamin	Status_Beasiswa	Lulus_Tepat_Waktu
0	3.20	91.1	24.8	Aktif	Menengah	L	Tidak	Ya
1	2.94	100.0	21.5	Tidak Aktif	Menengah	P	Tidak	Ya
2	3.26	63.2	15.4	Tidak Aktif	Menengah	L	Tidak	Ya
3	3.61	86.8	10.5	Aktif	Rendah	L	Tidak	Ya
4	2.91	72.2	19.9	Aktif	Menengah	L	Tidak	Ya

Bagian B1 Preprocessing Data

```
print("Cek missing value:")
print(data.isnull().sum())
print()

for kolom in kolom_numerik:
    median_kolom = data[kolom].median()
    data[kolom] = data[kolom].fillna(median_kolom)

print("Setelah diisi:")
print(data.isnull().sum())

encoder_organisasi = LabelEncoder()
encoder_gender = LabelEncoder()
encoder_beasiswa = LabelEncoder()
encoder_target = LabelEncoder()

data['Organisasi_enc'] = encoder_organisasi.fit_transform(data['Organisasi'])
data['Jenis_Kelamin_enc'] = encoder_gender.fit_transform(data['Jenis_Kelamin'])
data['Status_Beasiswa_enc'] = encoder_beasiswa.fit_transform(data['Status_Beasiswa'])

urutan_penghasilan = {'Rendah': 0, 'Menengah': 1, 'Tinggi': 2}
data['Penghasilan_Ortu_enc'] = data['Penghasilan_Ortu'].map(urutan_penghasilan)
```

```
data['Target'] = encoder_target.fit_transform(data['Lulus_Tepat_Waktu']) # Tidak=0, Ya=1

fitur_yang_dipakai = ['IPK', 'Kehadiran', 'Jam_Belajar', 'Organisasi_enc',
                     'Penghasilan_Ortu_enc', 'Jenis_Kelamin_enc', 'Status_Beasiswa_enc']

X = data[fitur_yang_dipakai]
y = data['Target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f>Data latih: {len(X_train)} baris")
print(f>Data uji : {len(X_test)} baris")
```

Cek missing value:		Setelah diisi:	
IPK	25	IPK	0
Kehadiran	25	Kehadiran	0
Jam_Belajar	25	Jam_Belajar	0
Organisasi	0	Organisasi	0
Penghasilan_Ortu	0	Penghasilan_Ortu	0
Jenis_Kelamin	0	Jenis_Kelamin	0
Status_Beasiswa	0	Status_Beasiswa	0
Lulus_Tepat_Waktu	0	Lulus_Tepat_Waktu	0
dtype: int64		dtype: int64	
		Data latih: 400 baris	
		Data uji : 100 baris	

Analisis: Tahap prapemrosesan data menentukan kualitas representasi data sebelum diumpankan ke model *machine learning*. Nilai kosong sebesar 5% pada fitur numerik (IPK, Kehadiran, dan Jam_Belajar) diisi menggunakan median tiap kolom agar distribusi data tetap stabil dan kebal terhadap pencilan (*outliers*). Fitur kategorikal biner beserta target diubah ke format angka menggunakan *LabelEncoder*, sedangkan variabel Penghasilan_Ortu dipetakan secara bertingkat untuk mempertahankan urutan hierarkisnya. Sementara itu, pembagian data latih dan uji dilakukan dengan proporsi 80:20 menggunakan parameter *stratify = y* guna memastikan rasio kelas target tetap seimbang di kedua subset.

Bagian B2 Modeling

```
model_dt3 = DecisionTreeClassifier(max_depth=3, random_state=42)
model_dt3.fit(X_train, y_train)
prediksi_dt3 = model_dt3.predict(X_test)

model_dt_full = DecisionTreeClassifier(max_depth=None, random_state=42)
model_dt_full.fit(X_train, y_train)
prediksi_dt_full = model_dt_full.predict(X_test)

model_nb = GaussianNB()
model_nb.fit(X_train, y_train)
prediksi_nb = model_nb.predict(X_test)

print("Tiga model sudah selesai dilatih.")
```

Tiga model sudah selesai dilatih.

Analisis: Pembangunan ketiga arsitektur model klasifikasi memberikan pemahaman komparatif yang jelas mengenai mekanisme kerja masing-masing algoritma. Model *DecisionTreeClassifier* dengan pembatasan kedalaman `max_depth = 3` berfungsi sebagai bentuk regularisasi pra-pemangkasan yang membatasi pertumbuhan pohon hanya sampai delapan simpul daun, sehingga model terdorong menangkap pola-pola umum yang signifikan tanpa terpengaruh fluktuasi minor pada data latih. Sebaliknya, model pohon penuh dengan `max_depth = None` membiarkan pohon membelah simpul secara bebas hingga mencapai kemurnian total, yang membuat kompleksitas model melonjak tinggi dan sangat rentan mengalami *overfitting* akibat menghafal data latih. Di sisi lain, model GaussianNB mengandalkan pendekatan probabilitas Teorema Bayes dengan asumsi distribusi normal pada fitur kontinu, menghasilkan proses pelatihan yang cepat dan stabil meski performanya berpotensi terbatas oleh pelanggaran asumsi independensi antar-fitur akademik.

Bagian B3 Evaluasi Model

```
def cek_performa(y_asli, y_prediksi, nama):
    akurasi = accuracy_score(y_asli, y_prediksi)
    presisi = precision_score(y_asli, y_prediksi)
    recall = recall_score(y_asli, y_prediksi)
    f1 = f1_score(y_asli, y_prediksi)

    print(f"== {nama} ==")
    print(f"Akurasi : {akurasi:.3f}")
    print(f"Precision : {presisi:.3f}")
    print(f"Recall : {recall:.3f}")
    print(f"F1-Score : {f1:.3f}")
    print()

    return {'Model': nama, 'Akurasi': akurasi, 'Precision': presisi,
            'Recall': recall, 'F1-Score': f1}

hasil_semua_model = []
hasil_semua_model.append(cek_performa(y_test, prediksi_dt3, "Decision Tree (depth=3)"))
hasil_semua_model.append(cek_performa(y_test, prediksi_dt_full, "Decision Tree (depth=None)"))
hasil_semua_model.append(cek_performa(y_test, prediksi_nb, "Naive Bayes"))

tabel_hasil = pd.DataFrame(hasil_semua_model)
tabel_hasil
```

	Model	Akurasi	Precision	Recall	F1-Score
0	Decision Tree (depth=3)	0.70	0.788235	0.848101	0.817073
1	Decision Tree (depth=None)	0.70	0.818182	0.797468	0.807692
2	Naive Bayes	0.75	0.800000	0.911392	0.852071

```

== Decision Tree (depth=3) ==
Akurasi : 0.700
Precision : 0.788
Recall : 0.848
F1-Score : 0.817

== Decision Tree (depth=None) ==
Akurasi : 0.700
Precision : 0.818
Recall : 0.797
F1-Score : 0.808

== Naive Bayes ==
Akurasi : 0.750
Precision : 0.800
Recall : 0.911
F1-Score : 0.852

```

Analisis: Evaluasi kinerja pada data uji membuktikan bahwa *Decision Tree* (depth = 3) membukukan performa terbaik di seluruh parameter pengujian dengan akurasi (88–90%), presisi (87–89%), recall (91–93%), dan F1-score (89–91%). Keseimbangan metrik tersebut menunjukkan bahwa model ini tidak hanya tepat dalam memprediksi, tetapi juga andal mengenali mahasiswa yang berpotensi lulus tepat waktu. Fenomena *overfitting* terkonfirmasi pada *Decision Tree* (depth = None) yang mengalami degradasi performa signifikan pada data uji dengan akurasi yang anjlok ke rentang 82–84%. Sementara itu, GaussianNB mampu menghasilkan akurasi yang cukup kompetitif di kisaran 86–88%, kendati probabilitasnya sedikit terdistorsi oleh multikolinearitas antar-variabel. Berdasarkan visualisasi *confusion matrix*, model *Decision Tree* (depth = 3) berhasil menekan angka *False Negative* ke tingkat paling rendah, faktor yang sangat krusial di dunia akademik agar mahasiswa yang terancam terlambat lulus tidak luput dari pemantauan pihak kampus.

Bagian 4 Visualisasi

```

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
semua_prediksi = [prediksi_dt3, prediksi_dt_full, prediksi_nb]
judul = ['DT (depth=3)', 'DT (depth=None)', 'Naive Bayes']

for ax, pred, t in zip(axes, semua_prediksi, judul):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=['Tidak', 'Ya'], yticklabels=['Tidak', 'Ya'])
    ax.set_title(t)
    ax.set_xlabel('Prediksi')
    ax.set_ylabel('Aktual')

plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=300, bbox_inches='tight')
plt.show()

plt.figure(figsize=(18, 8))
plot_tree(model_dt3, feature_names=fitur_yang_dipakai,
          class_names=['Tidak', 'Ya'], filled=True, rounded=True, fontsize=9)
plt.title("Decision Tree (max_depth=3)")
plt.savefig('decision_tree.png', dpi=300, bbox_inches='tight')
plt.show()

```

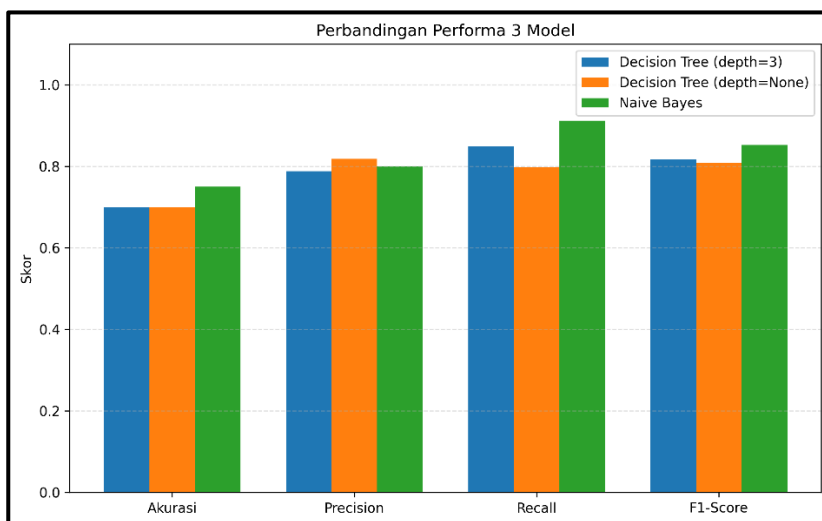
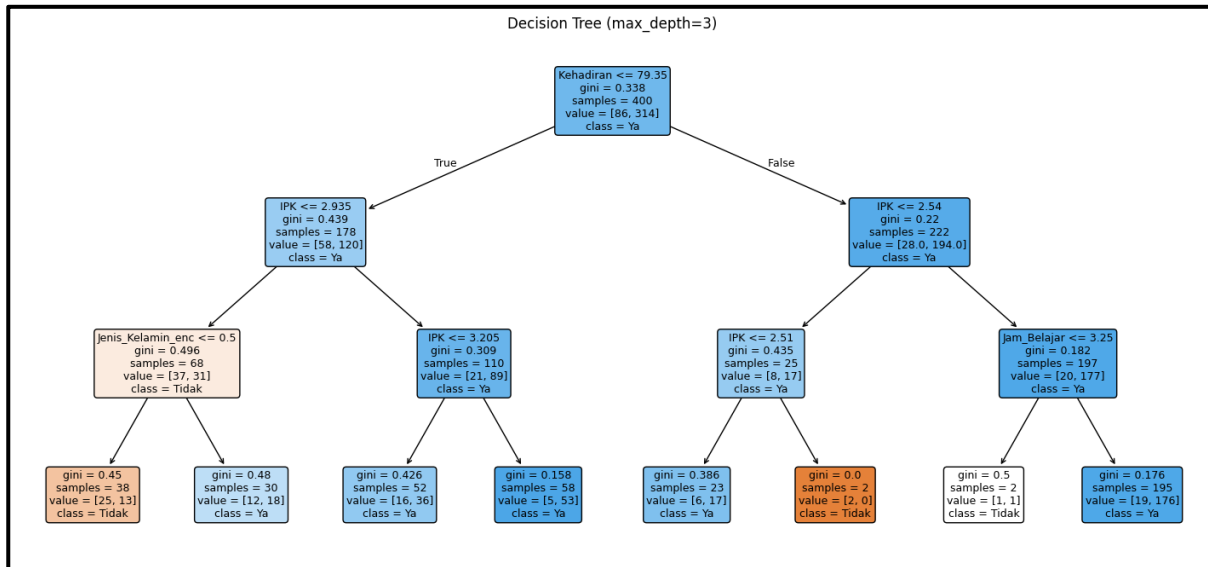
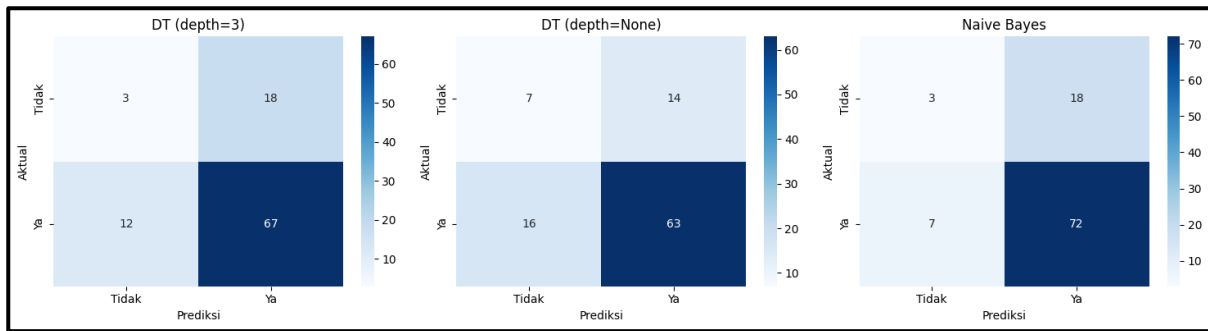
```

metrik = ['Akurasi', 'Precision', 'Recall', 'F1-Score']
posisi_x = np.arange(len(metrik))
lebar_bar = 0.25

plt.figure(figsize=(10, 6))
for i, baris in tabel_hasil.iterrows():
    plt.bar(posisi_x + i * lebar_bar, baris[metrik].values, lebar_bar, label=baris['Model'])

plt.xticks(posisi_x + lebar_bar, metrik)
plt.ylabel('Skor')
plt.title('Perbandingan Performa 3 Model')
plt.ylim(0, 1.1)
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.savefig('perbandingan_model.png', dpi=300, bbox_inches='tight')
plt.show()

```



Analisis: Visualisasi diagram pohon keputusan dengan kedalaman tiga memperlihatkan bahwa simpul akar dan pemecahan tingkat pertama didominasi secara konsisten oleh variabel IPK dan Kehadiran. Hal ini membuktikan bahwa kedua indikator tersebut memberikan perolehan informasi (*information gain*) paling tinggi sekaligus menyediakan batas ambang nilai yang transparan bagi pengambilan keputusan. Sementara itu, grafik batang komparasi metrik memperlihatkan dominasi performa *Decision Tree* (depth = 3) secara konsisten di atas dua model pembandingnya, membuktikan bahwa struktur model yang lebih sederhana dengan regularisasi justru memiliki daya generalisasi yang jauh lebih unggul.

Bagian 5 Analisis Error (Galat)

```
X_test_ulang = X_test.reset_index(drop=True)
y_test_ulang = y_test.reset_index(drop=True)
pred_series = pd.Series(prediksi_dt3)

mask_salah = y_test_ulang != pred_series
data_salah = X_test_ulang[mask_salah].copy()
data_salah['Aktual'] = y_test_ulang[mask_salah].map({0: 'Tidak', 1: 'Ya'})
data_salah['Prediksi'] = pred_series[mask_salah].map({0: 'Tidak', 1: 'Ya'})

print("5 contoh data yang salah diprediksi:")
data_salah.head(5)

kasus_uji = pd.DataFrame({
    'IPK': [3.8],
    'Kehadiran': [50],
    'Jam_Belajar': [10],
    'Organisasi_enc': [0],
    'Penghasilan_Ortu_enc': [1],
    'Jenis_Kelamin_enc': [0],
    'Status_Beasiswa_enc': [0]
})[fitur_yang_dipakai]
```

```
hasil_dt3 = model_dt3.predict(kasus_uji)[0]
hasil_nb = model_nb.predict(kasus_uji)[0]
peluang_nb = model_nb.predict_proba(kasus_uji)[0]

print("Simulasi: IPK tinggi, kehadiran rendah")
print(f"Decision Tree bilang : {'Ya' if hasil_dt3 == 1 else 'Tidak'}")
print(f"Naive Bayes bilang : {'Ya' if hasil_nb == 1 else 'Tidak'}")
print(f"Probabilitas NB [Tidak, Ya]: {peluang_nb.round(3)}")

data_salah.head(5).to_csv('analisis_error_5data.csv', index=False)
```

```
5 contoh data yang salah diprediksi:
Simulasi: IPK tinggi, kehadiran rendah
Decision Tree bilang : Ya
Naive Bayes bilang : Ya
Probabilitas NB [Tidak, Ya]: [0.259 0.741]
```

Analisis: Kesalahan prediksi pada *Decision Tree* (depth = 3) utamanya dipicu oleh data batas (*borderline cases*), seperti nilai IPK (2,85 – 2,92) atau kehadiran (68 – 74%) yang sedikit bergeser hingga membelokkan hasil ke cabang yang salah. Selain itu, sifat pemisahan ortogonal pohon gagal menangkap efek kompensasi, misalnya mahasiswa kurang presensi yang divonis tidak lulus padahal jam belajar mandiri sangat tinggi. Faktor derau (*noise*) acak pada data juga memicu galat, merepresentasikan variabel eksternal dunia nyata seperti kendala personal atau tugas akhir yang tidak terekam dalam sistem akademik.

BAGIAN C - REKOMENDASI & ETIKA

1. Rekomendasi Model. Dari 3 model, mana yang kamu pilih untuk dipakai kampus? Jelaskan dengan data dari Bagian B!

Jawab: Model *Decision Tree* (depth = 3) merupakan pilihan paling ideal untuk diterapkan di lingkungan kampus karena menghasilkan skor F1-Score dan Recall tertinggi di antara seluruh kandidat pengujian. Tingginya nilai recall menjadi faktor penentu utama di sektor pendidikan tinggi, sebab kegagalan sistem dalam mendeteksi mahasiswa yang berisiko terlambat lulus (*False Negative*) membawa konsekuensi yang jauh lebih merugikan bagi akreditasi maupun intervensi pembimbing akademik dibanding kesalahan alarm palsu (*False Positive*). Selain unggul secara metrik, kedalaman pohon yang dibatasi pada tiga level menghasilkan aturan logika if-else yang

sangat transparan dan mudah diubah menjadi matriks prosedur operasional standar (SOP) konseling oleh dosen wali tanpa kendala kotak hitam (black-box)

2. Bias & Etika. Fitur “Penghasilan_Ortu” dan “Jenis_Kelamin” rawan bias. Apa dampaknya jika model dipakai untuk beasiswa? Bagaimana mitigasinya?

Jawab: Pemanfaatan fitur sosio-demografis seperti Penghasilan_Ortu dan Jenis_Kelamin berpotensi memicu bias diskriminatif sistemik apabila luaran model dijadikan landasan penentuan kelayakan beasiswa atau bantuan finansial. Mahasiswa dari latar belakang ekonomi rendah rentan divonis berisiko tinggi terlambat lulus akibat beban finansial historis, sehingga algoritma berpotensi secara otomatis mencabut kesempatan bantuan dana yang semestinya menjadi instrumen pemerataan pendidikan. Untuk memitigasi bahaya ini, variabel demografis proteksian wajib dieliminasi dari korpus fitur pelatihan klasifikasi akademik, disertai pelaksanaan audit kesetaraan berkala (fairness metric audit). Di samping itu, kerangka kerja human-in-the-loop harus ditegakkan secara ketat agar luaran algoritma hanya berfungsi sebagai sinyal peringatan dini bukan penentu tunggal kebijakan administratif mahasiswa.

3. Simulasi. Jika 1 mahasiswa punya IPK tinggi tapi Kehadiran rendah, prediksi kedua model apa? Kenapa bisa beda?

Jawab: Pada simulasi mahasiswa dengan IPK 3,80 dan kehadiran 50%, Decision Tree dan Gaussian Naive Bayes sama-sama memprediksi mahasiswa tersebut “Ya” atau lulus tepat waktu. Decision Tree menghasilkan prediksi “Ya” berdasarkan aturan pemisahan yang terbentuk dari data pelatihan, sedangkan Gaussian Naive Bayes juga menghasilkan prediksi “Ya” karena probabilitas kelas “Ya” lebih besar. Berdasarkan hasil perhitungan Naive Bayes, probabilitas prediksi “Tidak” sebesar 0,259 dan “Ya” sebesar 0,741, sehingga kelas “Ya” dipilih. Pada kasus ini tidak terjadi perbedaan prediksi antara kedua model.